1. Crear carpeta para el proyecto : Proyecto_Final

2. Dentro de ella: Crear_pacientes.py
   primero podemos tener datos en python y luego usar json.dump() para crear el archivo Json

   Aquí escribimos los 5 pctes como una lista de diccionarios

   y luego usamos With open ("Pacientes.json", "w") as archivo:
   json.dump (pacientes, archivo, indent = 4)

   Esto hace que python tome la lista pacientes y la escriba en un archivo JSON

3. Entonces, la función del archivo Crear_pacientes.py
   es crear el archivo pacientes json

4. Aquí aparece Pacientes.json ⟶ Es el archivo donde van a quedar guardados permanentemente los pctes

5. Creamos Proyecto_final.py
   Es nuestro archivo ppal Donde vamos a construir el programa. Sirve para ejecutar todo el sistema.

6. Creamos el Readme.md
   Contiene la INFO del proyecto. Qué es, qué hace, qué archivos incluye. Cómo ejecutarlo

Guardamos los pctes en una lista llamada pctes
y cada pcte es un diccionario. Los numeros enteros (edad) no tiene que ir en comillas. Si no, python lo toma como texto y no haría cálculos con él.

> "Primero creamos una carpeta para organizar el proyecto. Después creamos un archivo llamado `crear_pacientes.py`, donde escribimos los pacientes ficticios como una lista de diccionarios. Con `json.dump()` generamos automáticamente el archivo `pacientes.json`. Ese archivo queda como almacenamiento permanente de los pacientes. Después creamos `proyecto_final.py`, que es el programa principal y que lee ese JSON para poder trabajar con los pacientes. Finalmente creamos `README.md`, que explica el proyecto y cómo ejecutarlo."

proyecto_final.py | crear_pacientes.py | pacientes.json | README.md

crear_pacientes.py >

```python
import json


pacientes = [
    {
        "id": "P001",
        "nombre": "Victor Rivera",
        "edad": 59,
        "sexo": "F",
        "usa_correccion": True
    },
    {
        "id": "P002",
        "nombre": "Sandra Rengifo",
        "edad": 56,
        "sexo": "F",
        "usa_correccion": False
    },
    {
        "id": "P003",
        "nombre": "Lina Rivera",
        "edad": 33,
        "sexo": "F",
        "usa_correccion": True
    },
    {
        "id": "P004",
        "nombre": "Alejandro Rivera",
        "edad": 20,
        "sexo": "M",
        "usa_correccion": True
    },
    {
        "id": "P005",
        "nombre": "Johanna Rivera",
        "edad": 23,
        "sexo": "F",
        "usa_correccion": False
    }
]


with open("pacientes.json", "w") as archivo:

    json.dump(
        pacientes,
        archivo,
        indent=4
    )

```

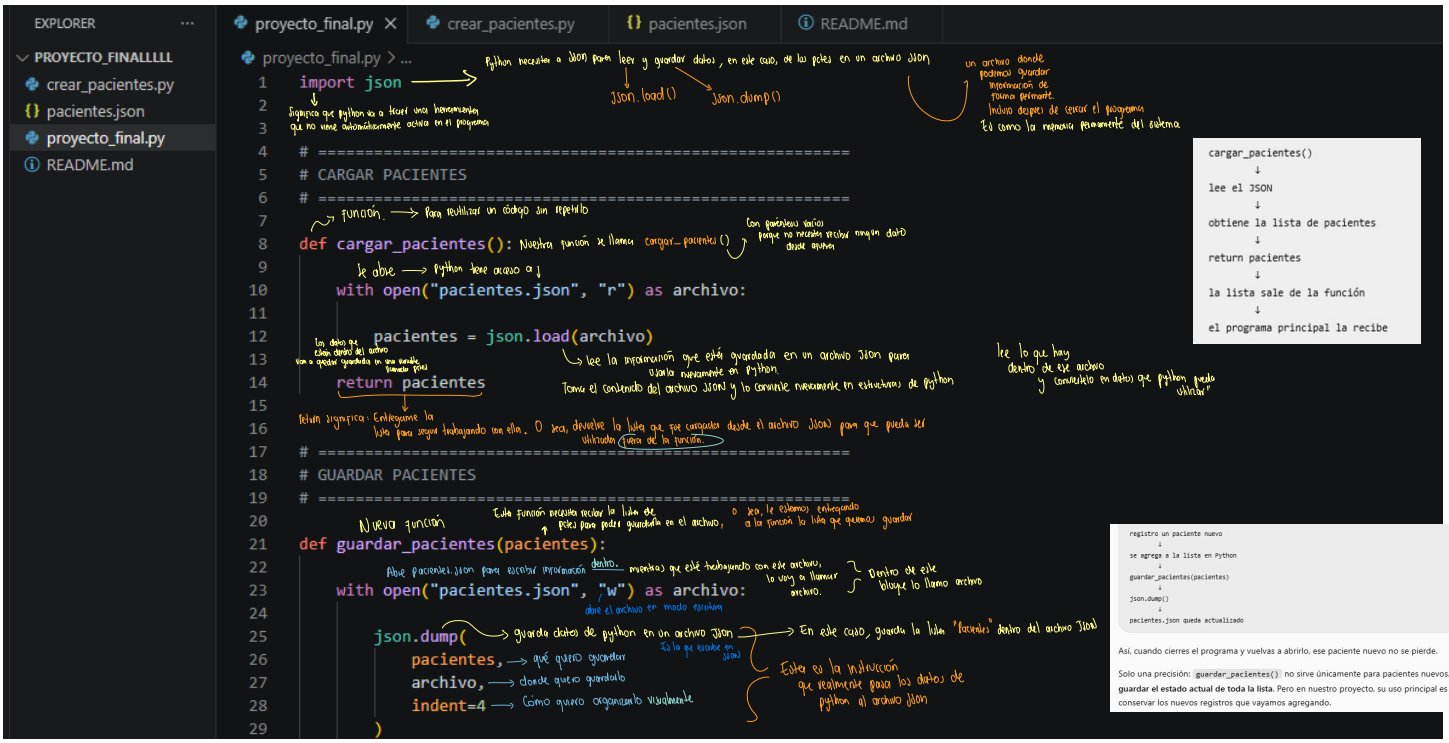
crea

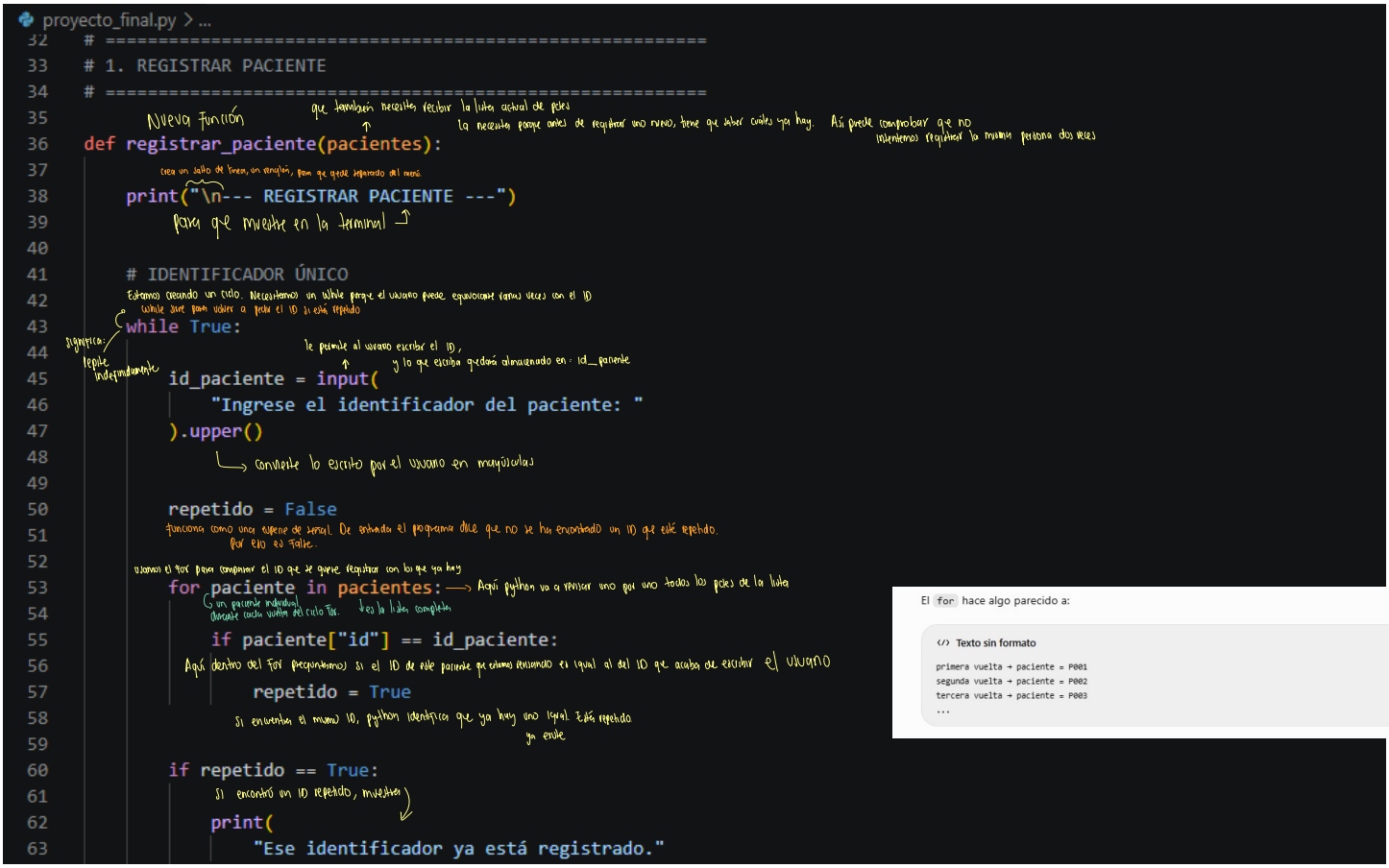
`crear_pacientes.py` crea los datos iniciales.

`pacientes.json` los conserva.

Y `proyecto_final.py` los utiliza.

> "El archivo JSON funciona como el lugar donde quedan almacenados los pacientes. Primero creamos los datos en Python y con `json.dump()` los guardamos en `pacientes.json`. Después el programa principal puede volver a leerlos con `json.load()`, por eso los pacientes no desaparecen cuando cerramos el programa."


EXPLORER
PROYECTO_FINALLLLL
crear_pacientes.py
pacientes.json
proyecto_final.py
README.md
proyecto_final.py
crear_pacientes.py
pacientes.json
README.md
import json
CARGAR PACIENTES
def cargar_pacientes():
with open("pacientes.json", "r") as archivo:
pacientes = json.load(archivo)
return pacientes
GUARDAR PACIENTES
def guardar_pacientes(pacientes):
with open("pacientes.json", "w") as archivo:
json.dump(
pacientes,
archivo,
indent=4
)
cargar_pacientes()
lee el JSON
obtiene la lista de pacientes
return pacientes
la lista sale de la función
el programa principal la recibe
registro un paciente nuevo
se agrega a la lista en Python
guardar_pacientes(pacientes)
json.dump()
pacientes.json queda actualizado
Así, cuando cierres el programa y vuelvas a abrirlo, ese paciente nuevo no se pierde.
Solo una precisión: guardar_pacientes() no sirve únicamente para pacientes nuevos. En general sirve para guardar el estado actual de toda la lista. Pero en nuestro proyecto, su uso principal es precisamente conservar los nuevos registros que vayamos agregando.


proyecto_final.py
1. REGISTRAR PACIENTE
def registrar_paciente(pacientes):
print("\n--- REGISTRAR PACIENTE ---")
IDENTIFICADOR ÚNICO
while True:
id_paciente = input(
"Ingrese el identificador del paciente: "
).upper()
repetido = False
for paciente in pacientes:
if paciente["id"] == id_paciente:
repetido = True
if repetido == True:
print(
"Ese identificador ya está registrado."
El for hace algo parecido a:
Texto sin formato
primera vuelta → paciente = P001
segunda vuelta → paciente = P002
tercera vuelta → paciente = P003
...

```
proyecto_final.py > registrar_paciente
36  def registrar_paciente(pacientes):
                print(
                    "Ese identificador ya está registrado."
                )  Y como todavía estamos dentro del ciclo while True, python vuelve arriba y solicita otro ID.

            else: → Esto significa, si el ID no estaba repetido, o sea, que sigue siendo "False"
                        entonces se rompe el ciclo para continuar con el resto del registro
                break


        # NOMBRE      le pide al usuario que escriba algo en la consola
                      ↑   ↓
        nombre = input(
            "Ingrese el nombre completo: "
        )  lo que escriba, lo guarda en la variable nombre

        # EDAD
                  ↗ para convertir lo que ingrese el usuario en un número entero
        edad = int(
            input("Ingrese la edad: ")
        )  le estamos diciendo que pida la edad, la convierta en un número entero y la guarde en
                                                la variable edad.

        while edad < 1 or edad > 120:
            mientras que la edad sea menor que 1 o mayor que 120, sigue repitiendo
            print(
                "La edad debe estar entre 1 y 120 años."
            )          muestra el mensaje
```

```
36  def registrar_paciente(pacientes):
            )
                        Vuelve a pedir la edad, como aquí el input no estaba dentro del while,
            edad = int(                                    debemos escribir un segundo input.
                input("Ingrese nuevamente la edad: ")          para que el valor pueda cambiar.
            )
                Al ingresar la edad correcta, se rompe el
                                    ciclo solo.

        # SEXO

        sexo = input(
            "Ingrese el sexo: "
        ).upper()
             → mayúsculas

        # CORRECCIÓN VISUAL
        → usamos otra vez while True porque queremos que el programa siga preguntando hasta que el usuario escriba una respuesta válida
        while True:
                Primero pedimos la respuesta (si/no)
            respuesta = input(
                "¿Usa corrección visual? (si/no): "
            ).lower()
                 → convierte la rpta en minúsculas

            if respuesta == "si":
                si la rpta es "si"
                usa_correccion = True
                    Guardamos el sí como un booleano que sería True,
                break          y ahí se rompe el ciclo.
```

Porque el profesor nos pide que esa info se maneje con booleanos

```python
def registrar_paciente(pacientes):
        # Si la respuesta es "no"
        elif respuesta == "no":
            # Se guarda como falso
            usa_correccion = False
            # Se rompe el ciclo.
            break

        # Si el pete no escribió ni sí ni no,
        else:

            print(
                "Debe responder si o no."
            )


    # CREAR DICCIONARIO ⟶ Es donde por fin reunimos toda la info. que acabamos de pedirle al usuario.
    # En la variable "nuevo pete" vamos a guardar un diccionario
    nuevo_paciente = {

        "id": id_paciente,
        # la clave sería el nombre que usamos para identificar el dato  ↳ sería la variable que contiene el dato real que escribió el usuario
        "nombre": nombre,

        "edad": edad,

        "sexo": sexo,

        "usa_correccion": usa_correccion
    }
```

Hasta el momento teníamos todos los datos separados en distintas variables sueltas.
Python sabe que existen esos datos, pero necesitamos agruparlos para decirle que todo pertenece al mismo paciente.

Contiene lo que haya escrito el usuario en ese momento

```python
def registrar_paciente(pacientes):
    # Ya tenemos nuevo pete, ahora lo metemos dentro de la lista que ya teníamos usando append
    # AGREGAR A LA LISTA
    pacientes.append(
        nuevo_paciente
    )

    # GUARDAR EN JSON
    # Llamamos la función que creamos al inicio
    guardar_pacientes(
        pacientes
    )
        # Necesitamos que el pete que acabamos de registrar quede guardado permanentemente en el archivo JSON.


    print(
        "Paciente registrado correctamente."
    )
        # Es lo que aparecerá luego de haber creado el diccionario

# aparece como confirmación de que el diccionario se agregó a la lista y se guardó en JSON.

# ============================================================
# 2. CONSULTAR PACIENTE
# ============================================================
# Función que nos va a servir para mostrar un pete que esté registrado
def consultar_paciente(pacientes):
    # ⟶ llamamos a la lista porque necesita acceso a ella para poder buscar

    print("\n--- CONSULTAR PACIENTE ---")
```

```python
# ============================================================
# 2. CONSULTAR PACIENTE
# ============================================================

def consultar_paciente(pacientes):

    print("\n--- CONSULTAR PACIENTE ---")
              Para que le muestre en la consola

    id_buscar = input(
        "Ingrese el identificador del paciente: "
    ).upper()  Aquí le pedimos al usuario que escriba el ID del pcte que quiere consultar
        ↳ mayúsculas

    encontrado = False
Volvemos a usar una variable como bandera
        Aquí asumimos que aún no hemos encontrado al pcte.

    for paciente in pacientes:
     Usamos ciclo For para recorrer uno por uno todos los pctes , y en cada vuelta comparar el id que le acaba de escribir con los que ya hay registrados
        if paciente["id"] == id_buscar:
            Si es igual, entonces encontró el paciente, y comienza a mostrar todos
                                                    sus datos.
            print(
                "ID:",
                paciente["id"]
            )

            print(
                "Nombre:",
                paciente["nombre"]
            )
```

```python

            print(
                "Edad:",
                paciente["edad"]
            )

            print(
                "Sexo:",
                paciente["sexo"]
            )

            print(
                "Usa corrección visual:",
                paciente["usa_correccion"]
            )


            encontrado = True
                    ↳ Cuando se encontró se
                      cambia la bandera a True y se rompe el ciclo For
                                para que no siga revisando
            break

    if encontrado == False:
                Si después de haber revisado la lista, no se encuentra
                    nada, entonces:
        print(
            "El paciente no está registrado."
        )
```

Lo que estamos haciendo es acceder a cada dato del diccionario usando sus claves

> "La función `consultar_paciente` recibe la lista de pacientes y pide el ID que se desea buscar. Inicialmente usamos la variable `encontrado` en `False`. Luego recorremos la lista con un `for` y comparamos cada ID con el ingresado. Si encontramos una coincidencia, mostramos los datos del paciente, cambiamos `encontrado` a `True` y usamos `break` porque ya no necesitamos seguir buscando. Si al terminar el recorrido `encontrado` continúa en `False`, mostramos que el paciente no está registrado."

```python
# ============================================================
# 3. MOSTRAR PACIENTES REGISTRADOS
# ============================================================

def mostrar_pacientes(pacientes):
    print(
        "\n--- PACIENTES REGISTRADOS ---"
    )

    if len(pacientes) == 0:
        print(
            "No hay pacientes registrados."
        )

    else:
        for paciente in pacientes:
            print(
                paciente["nombre"]
            )
```

"Esta función recibe la lista de pacientes. Primero verifica con `len()` si la lista está vacía. Si no hay pacientes, muestra un mensaje. Si sí hay, utiliza un `for` para recorrer cada diccionario y mostrar únicamente el nombre de cada paciente."

proyecto_final.py > registrar_paciente

```python
# ============================================================
# 4. BUSCAR PACIENTES CON CORRECCIÓN VISUAL
# ============================================================

def buscar_correccion(pacientes):

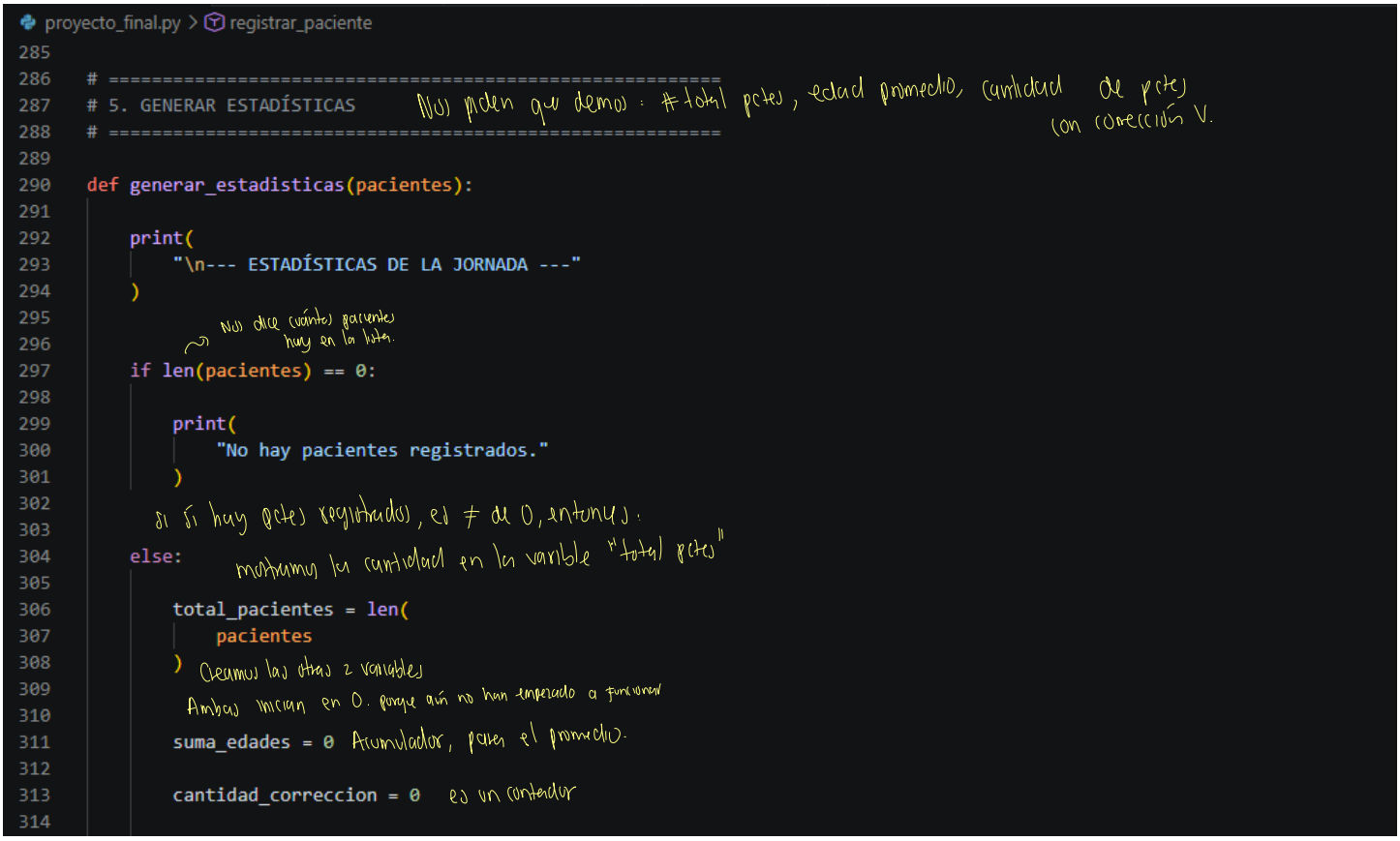
    print(
        "\n--- PACIENTES CON CORRECCIÓN VISUAL ---"
    )

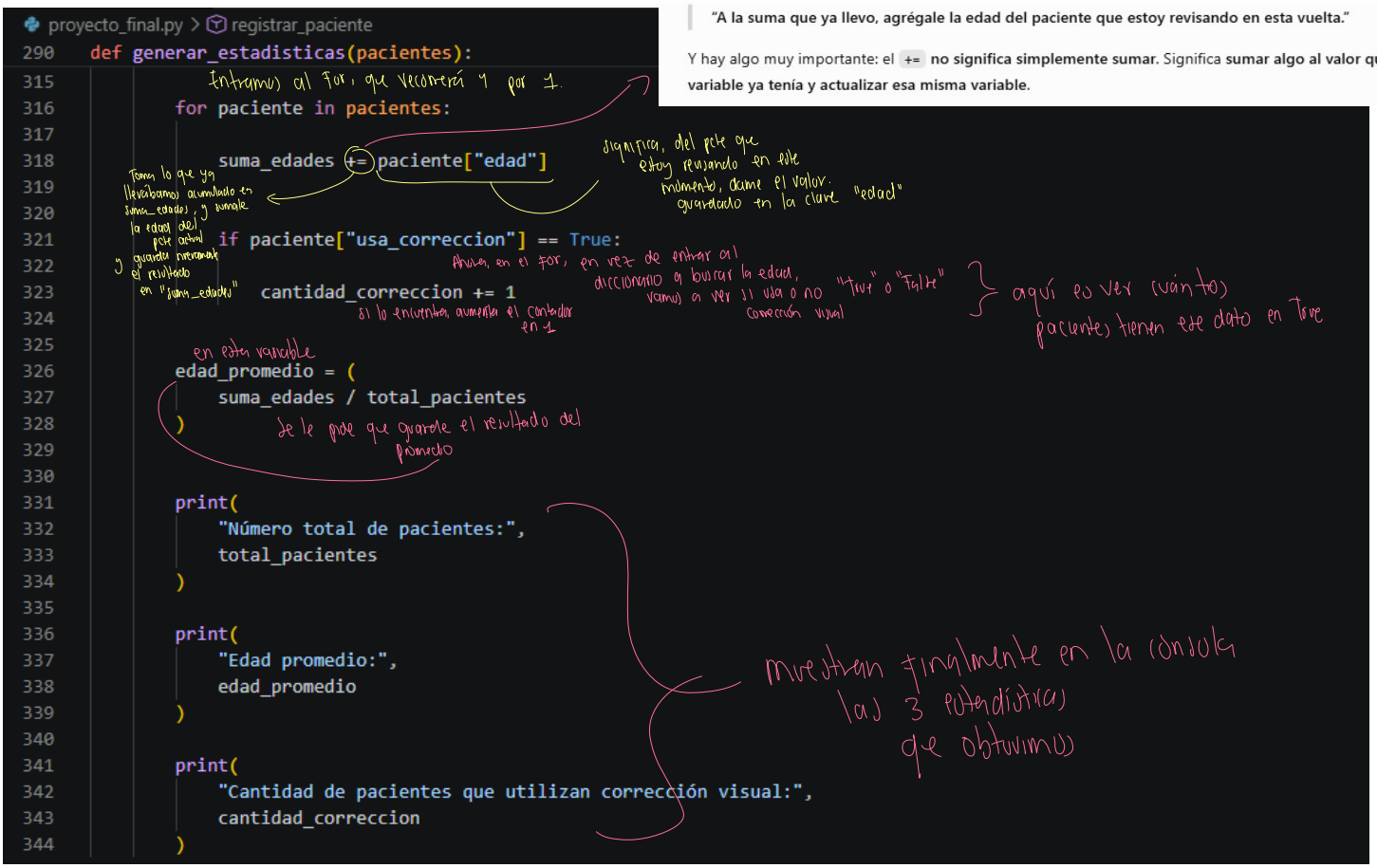
    encontrados = 0

    for paciente in pacientes:
        if paciente["usa_correccion"] == True:
            print(
                paciente["nombre"]
            )
            encontrados += 1

    if encontrados == 0:

        print(
            "No hay pacientes que utilicen corrección visual."
        )
```





> "A la suma que ya llevo, agrégale la edad del paciente que estoy revisando en esta vuelta."

Y hay algo muy importante: el `+=` **no significa simplemente sumar.** Significa **sumar algo al valor que la variable ya tenía y actualizar esa misma variable.**

```python
# ============================================================
# MENÚ PRINCIPAL
# ============================================================

def menu():
    print("\n====================================")

    print("        SISTEMA OPTOGESTIÓN")

    print("====================================")

    print("1. Registrar paciente")

    print("2. Consultar paciente")

    print("3. Mostrar pacientes registrados")

    print("4. Buscar pacientes con corrección visual")

    print("5. Generar estadísticas")

    print("6. Salir")


    opcion = input(
        "\nSeleccione una opción: "
    )

```

el trabajo de esta variable es mostrar las opciones del programa y recibir la opción que el usuario quiere ejecutar

Esta función no necesita recibir ningún dato para funcionar.

Aquí solo necesitan mostrar textos y preguntarle al usuario qué quiere hacer

mostrar la información

Para permitirle al usuario escribir la opción

input recibe texto y lo guarda en la variable "opcion"

Entonces la función `menu()` hace dos cosas esenciales:

> **muestra las opciones del sistema y devuelve la opción seleccionada por el usuario.**

Y aquí `return` vuelve a ser importante porque no queremos simplemente mostrar la opción en pantalla; queremos **entregarla al resto del programa para que sepa qué función ejecutar.**

```python
proyecto_final.py > registrar_paciente
    def menu():

        return opcion


# ============================================================
# PROGRAMA PRINCIPAL  Donde por fin se conectan todas las funciones que construimos antes.
# ============================================================

pacientes = cargar_pacientes()
" Carga los pacientes del archivo JSON y guarda la lista que devuelve la función en una

while True:
    Es el que va a mantener todo el programa funcionando continuamente
    opcion = menu()

    if opcion == "1":
        registrar_paciente(
            pacientes
        )


    elif opcion == "2":

        consultar_paciente(
            pacientes
        )

```

Usamos return para entregarle esa respuesta al programa principal, para que él pueda saber qué hacer, le dice al programa la opción que escribió el usuario, porque hasta ahora estaba guardada dentro de la función

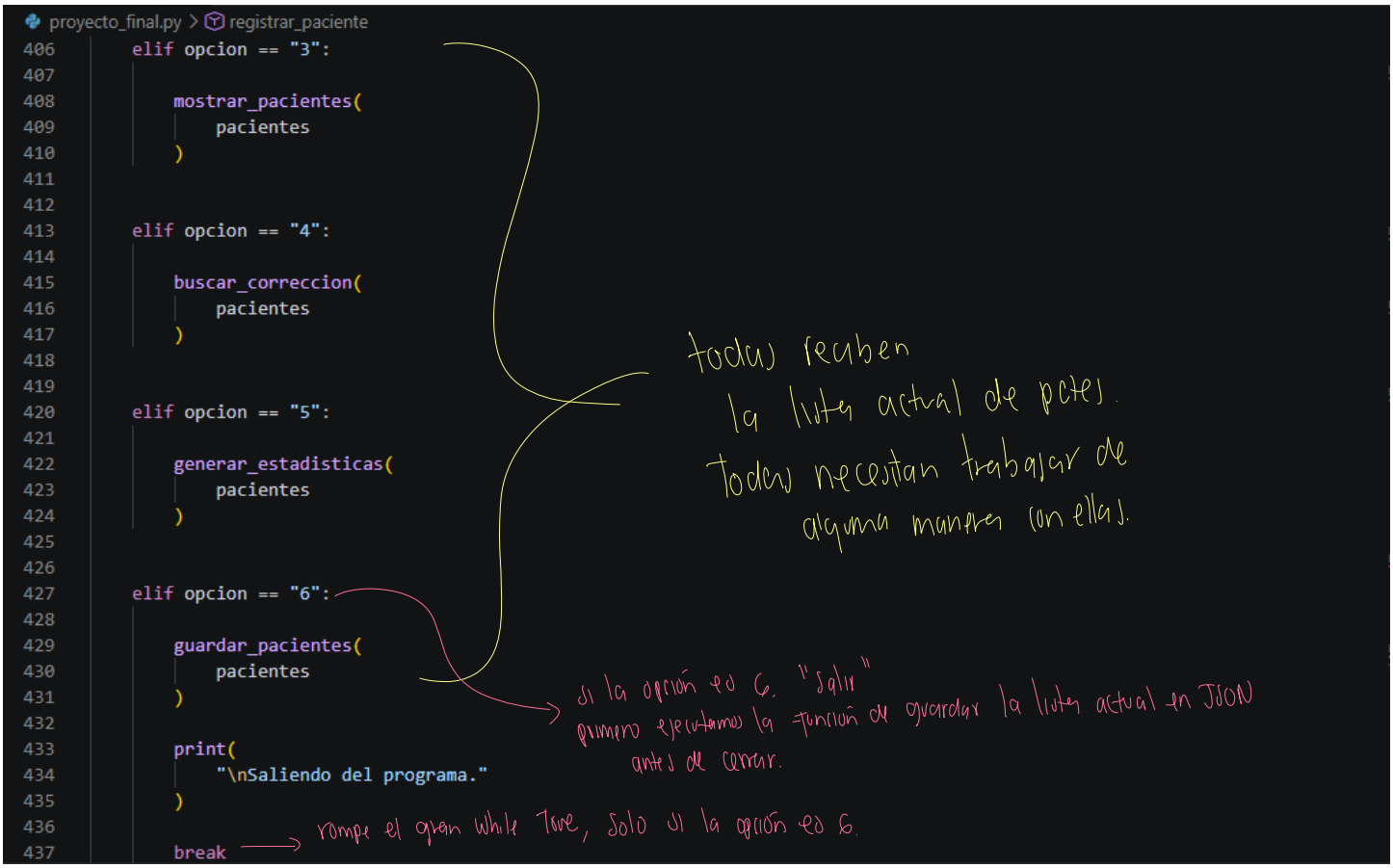
Aquí es donde se van a ejecutar las funciones que hicimos definido al principio

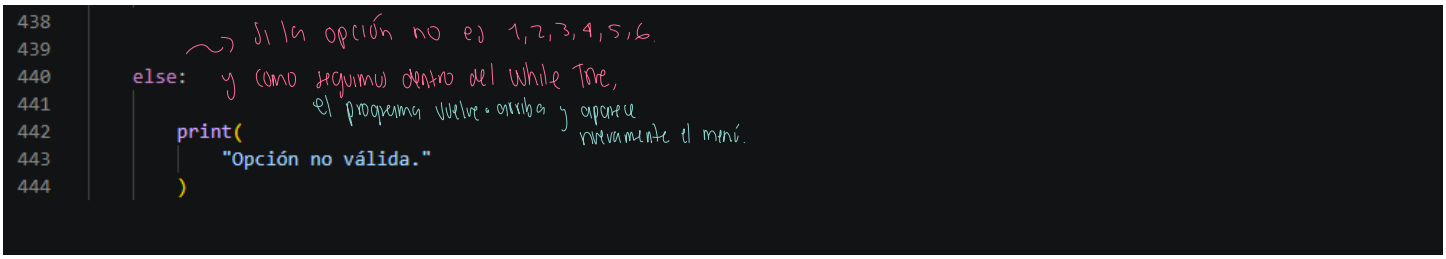
Variable llamada "pac". Esto es para que él tenga esta lista "preparada" para que las otras demás funcionen.

lo necesitamos porque no queremos que el programa termine después de hacer una sola cosa.

aquí estamos llamando la función de menú. para que muestre todas las opciones.

si el usuario escribió 1 : ejecuta la función registrar pcte





> "Primero se cargan desde el JSON los pacientes ya existentes mediante `cargar_pacientes()`. Después comienza un `while True` que mantiene activo el programa. En cada repetición se ejecuta `menu()`, que devuelve la opción seleccionada. Según esa opción, una cadena de `if` y `elif` llama a la función correspondiente. Después de ejecutar una opción, el ciclo vuelve a mostrar el menú. Cuando el usuario elige 6, se guarda la lista actualizada, se muestra el mensaje de salida y `break` termina el ciclo y el programa."